Universidad Nacional Autónoma de México

Facultad de Ingeniería

Manual de algoritmos de ordenamiento y búsqueda

Estructura de Datos y Algoritmos II

Profesor: Edgar Tista García

Integrantes

Frías Hernández Edmond Antonio
Robles Cancino Arturo Andres
Zozaya Fernández Melanie

29 de Octubre del 2026

Introducción

El objetivo es que el alumno realice una solución de software implementando los algoritmos de ordenamiento más eficientes, y los algoritmos de búsqueda por comparación y transformación de llaves.

Desarrollo

Explicación general de la solución de software

En primera instancia, el problema se abordó usando la arquitectura Modelo-Vista-Controlador para dividir en tres grandes vertientes el desarrollo del software y sus funcionalidades.

Modelo

Este segmento es el encargado de tener las bases que permiten definir los objetos que usaremos durante la ejecución. Creamos nuestra estructura de datos, que es una tabla hash, y nuestra entidad principal, que usamos en todo el programa.

- Base de datos (estructura de datos)
- *Bug* (entidad principal)

Controlador

Contiene la lógica general de manipulación de datos. Aquí se alojan los algoritmos de búsqueda y ordenamiento personalizados, separando la lógica de la vista del usuario. Además, se gestionan las consultas de formas más específicas usando la algoritmia mencionada. Interactúa con los datos del Modelo para poder consultarlos.

Vista

Está el menú con el cual interactúa el usuario; permite las entradas y las redirige a lo necesario del controlador, así se mantiene toda la lógica aislada de lo que el usuario puede hacer.

Explicación de cómo se implementan los conceptos de ordenamiento

Tomando los requerimientos previstos en el problema, fue necesario tener presentes diferentes ordenamientos dependiendo del criterio que el usuario pudiera llegar a elegir, y así, internamente, para cada criterio se eligió un algoritmo. Los implementados son:

- **Marge Short (Merge sort)**: Para los criterios de título, ID y categoría se emplea este algoritmo por su gran facilidad para ordenar, siendo especialmente eficiente para cadenas de texto.
- **Counting sort**: Se aplicó a criterios numéricos como prioridad, nivel de seguridad y estado, dado que no sobrepasan el rango de 1 a 5; entonces resultan poco extensos en su uso de memoria y eficientes en ordenamiento, ya que no es un gran volumen de datos. Estas son las razones por las que se optó por su implementación.
- **Bubble sort**: Se empleó para los criterios de fecha y equipo, pensando en que bubble sort permite hacer un ordenamiento, aunque mucho peor en comparación con el resto de algoritmos, lo hace relativamente rápido cuando los datos están casi ordenados. Consideramos que el caso de las fechas es uno de esos casos donde puede resultar bueno su uso.

Explicación de cómo se implementan los conceptos de búsqueda

El software lo toma por dos vertientes principales, ya que se basa en la implementación de transformación de claves y su asignación en el contenedor que es la base de datos:

- **Búsqueda por transformación de claves**: Esta se realiza únicamente cuando el usuario selecciona el criterio de búsqueda por ID, ya que este funge casi como el índice de la base de datos; entonces no es complicado encontrar un ID y, de hecho, logramos una complejidad constante en la mayoría de los casos. Cuando hay colisiones, por alguna razón, se implementó como lista de listas, entonces se realiza una búsqueda lineal para esos casos.
- **Búsqueda binaria**: Tomando como base que todos los *bugs* se encuentran en la base de datos, se itera buscando las coincidencias para los campos título, categoría y equipo; así, cuando se termina de recolectar esto, internamente se ordena empleando los ordenamientos del controlador y, finalmente, se busca sobre este campo de forma relativamente rápida gracias a la búsqueda binaria.

Conclusiones

Frías Hernández Edmond Antonio: El uso de colecciones, métodos de ordenamiento y métodos de búsqueda combinados permiten crear programas más complejos que, junto a lo que ofrece Java (interfaces, clases, herencia, etc.), permiten plasmar lo aprendido en clase de formas distintas y creativas.

Robles Cancino Arturo Andres: El desarrollo de la práctica potenció la abstracción de un problema más real, donde es necesaria una implementación planeada sobre todo lo aprendido, buscando los caminos más cortos pero a la vez eficientes para la selección de algoritmos y estructuras de datos; sin duda, fue un gran refuerzo de lo aprendido hasta el momento.

Zozaya Fernández Melanie: El desarrollo del Administrador de Bugs me ayudó a complementar la aplicación de algoritmos de ordenamiento a distintos tipos de datos; también me ayudó a reforzar la arquitectura modular basada en paquetes para tener una separación limpia.

Referencias

[1] C. Álvarez Caules, *¿que es un java predicate?* Arquitectura Java, Blog post, 2017. [Online]. Available: https://www.arquitecturajava.com/java-predicate/

[2] GeeksforGeeks, *Java 8 predicate with examples*, GeeksforGeeks, Última actualización: 19 de junio de 2026, 2026. [Online]. Available: https://www.geeksforgeeks.org/java/java-8-predicate-with-examples/

[3] javapro.org, *Java: Pasar un método como parámetro*, javapro.org, Publicado el 25 de agosto de 2023, 2023. [Online]. Available: https://javapro.org/programming-languages/java-pasar-un-metodo-como-parametro/

[4] GeeksforGeeks, *Hash function for string data in c#*, GeeksforGeeks, Última actualización: 5 de noviembre de 2024, 2024. [Online]. Available: https://www.geeksforgeeks.org/c-sharp/hash-function-for-string-data-in-c-sharp/